

Algo!

An AI-Powered LeetCode Hint System

Chrome Extension + Cloud-Deployed RAG Backend

TypeScript · Express 5 · Google Gemini · Redis HNSW

Docker · Google Cloud Run · GitHub Actions

Nathan L.

April 2026

Abstract

Algo! is a Chrome extension that injects a floating chat interface into LeetCode problem pages, enabling students to receive on-demand, AI-generated hints without leaving their coding environment. A Node.js/Express backend orchestrates a Retrieval-Augmented Generation (RAG) pipeline built on Google Gemini (`gemini-2.5-flash` for generation, `gemini-embedding-001` for embeddings) and a Redis HNSW vector index for semantic problem retrieval. The backend also maintains a TTL-based Redis hint cache to minimise API latency and cost. The entire stack is containerised with Docker and deployed to Google Cloud Run via a GitHub Actions CI/CD pipeline, giving the service automatic scaling to zero and sub-second cold starts. This report describes the system architecture, implementation details, performance considerations, and the testing strategy used to validate the system.

1 Introduction

Competitive programming platforms such as LeetCode provide thousands of algorithmic problems for practice, yet the feedback loop is often slow: a student is stuck, searches for a hint, and risks seeing a complete solution that eliminates the learning benefit. **Algo!** addresses this gap by embedding an interactive chat assistant directly in the browser tab.

The assistant is deliberately *constrained*: the LLM prompt instructs the model to reply in one-to-three sentences, give a single actionable next step, and never provide a full solution. Contextual relevance is improved through a RAG mechanism that surfaces structurally similar past problems from a growing Redis vector index. Repeated hint requests for the same problem/question pair are served from a Redis hint cache, reducing Gemini API call volume and response latency.

The project was developed on a `ts_migration` branch, migrating the original JavaScript extension to a fully typed TypeScript codebase (both extension and backend), culminating in the clean final state described in this report.

2 System Architecture

Figure 1 illustrates the end-to-end data flow.

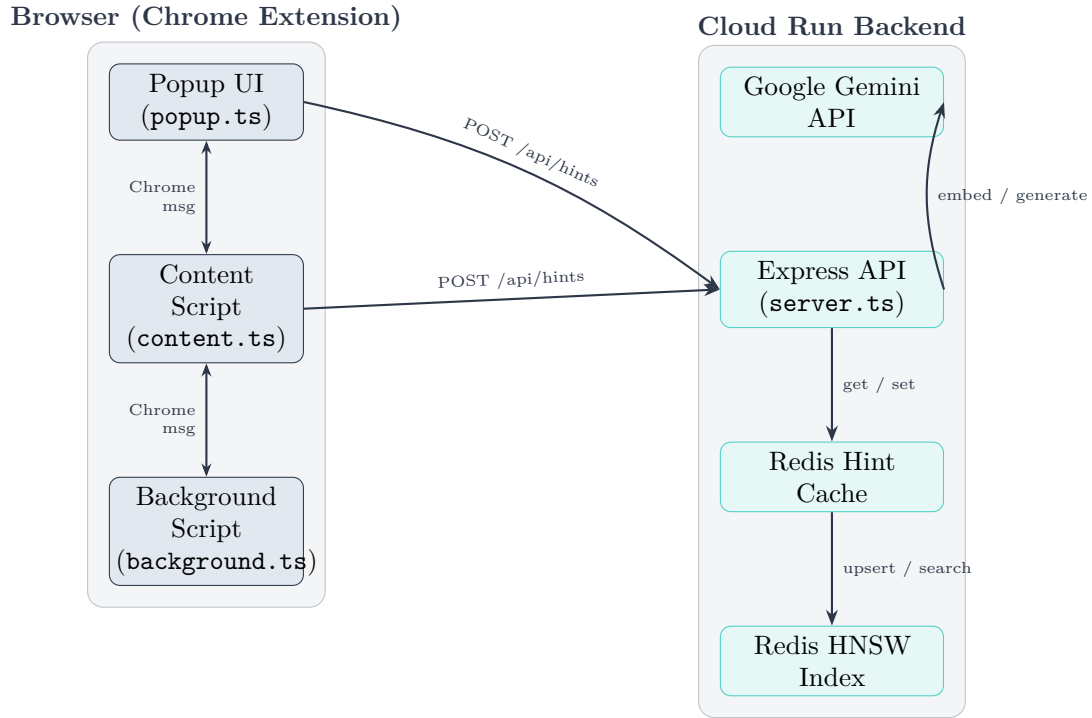


Figure 1: End-to-end system architecture. The Chrome extension has three scripts: the *popup* renders the UI, the *background* script relays Chrome messages, and the *content* script injects the floating chat widget and calls the backend directly.

2.1 Component Responsibilities

Component	Responsibility
<code>content.ts</code>	Injected into LeetCode pages; renders the floating chat FAB and drop-down; reads problem metadata from the DOM and Monaco editor; calls <code>/api/hints</code> .
<code>background.ts</code>	Service-worker relay; bridges Chrome messages between the popup and the active-tab content script.
<code>popup.ts</code>	Extension toolbar popup; shows problem title, difficulty, language, and tags; supports direct hint requests via the same backend endpoint.
<code>config.ts</code>	Declares <code>__ALGO_API_BASE_URL__</code> ambient global and exports <code>API_BASE_URL</code> with a <code>localhost:3000</code> fallback.
<code>server.ts</code>	Express 5 API; handles hint generation, problem indexing, and vector search; manages the hint cache and the HNSW vector store.
<code>vectorStore.ts</code>	Redis HNSW service; wraps <code>FT.CREATE</code> and <code>FT.SEARCH</code> via <code>ioredis.call()</code> .
<code>cache.ts</code>	Redis hint cache; wraps <code>SETEX/GET/DEL</code> for TTL-bound hint storage.

3 Chrome Extension

3.1 Build Pipeline

The extension source is written in TypeScript and compiled by **webpack 5** with **ts-loader**. Three entry points produce three output bundles:

Entry point	Output	Bundle size (gzip)
src/content.ts	dist/content.js	12.6 KiB
src/popup.ts	dist/popup.js	3.6 KiB
src/background.ts	dist/background.js	0.6 KiB

The `webpack.DefinePlugin` injects the environment variable `ALGO_API_BASE_URL` at build time, allowing the same codebase to target local development (`localhost:3000`) and the Cloud Run production URL without source changes.

3.2 Content Script UI

The content script appends two DOM elements to every matched LeetCode page:

1. A circular **FAB** (Floating Action Button) anchored to the bottom-right corner.
2. A **chat dropdown** (300 px × 450 px) with a message list, text area, and send button.

The chat uses CSS **animation** for message appearance and a three-dot typing indicator while awaiting the API response. The colour scheme uses teal (`#4FD1C5`) accents on a dark charcoal background (`#1A202C`) to minimise visual distraction during problem-solving.

3.3 Message Passing

Chrome Manifest V3 service workers handle message passing between scripts. Figure 2 shows the sequence for a popup hint request.

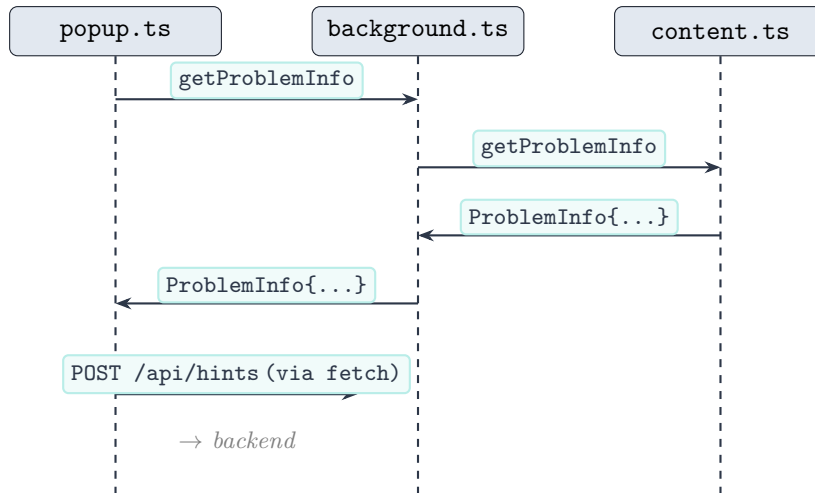


Figure 2: Chrome message-passing sequence for a popup hint request. The popup first obtains problem metadata from the active tab’s content script via the background service worker, then issues the HTTP request itself.

4 Backend API

4.1 Endpoints

Method	Path	Description
GET	/	Root info; lists all available endpoints.
GET	/healthz	Liveness probe; pings Redis and returns 200 ok.
GET	/readyz	Readiness probe; checks both Redis and GEMINI_API_KEY.
POST	/api/hints	Main hint endpoint; accepts <code>message</code> and optional <code>problemInfo</code> ; returns <code>{hint, source, retrievalCount}</code> .
POST	/api/index-problem	Embeds and upserts a problem document into the HNSW vector index.
POST	/api/rag/search	Accepts an arbitrary <code>query</code> string and returns the top- K most similar indexed problems.

4.2 Hint Request Lifecycle

Figure 3 traces the decision path for a single POST `/api/hints` request.

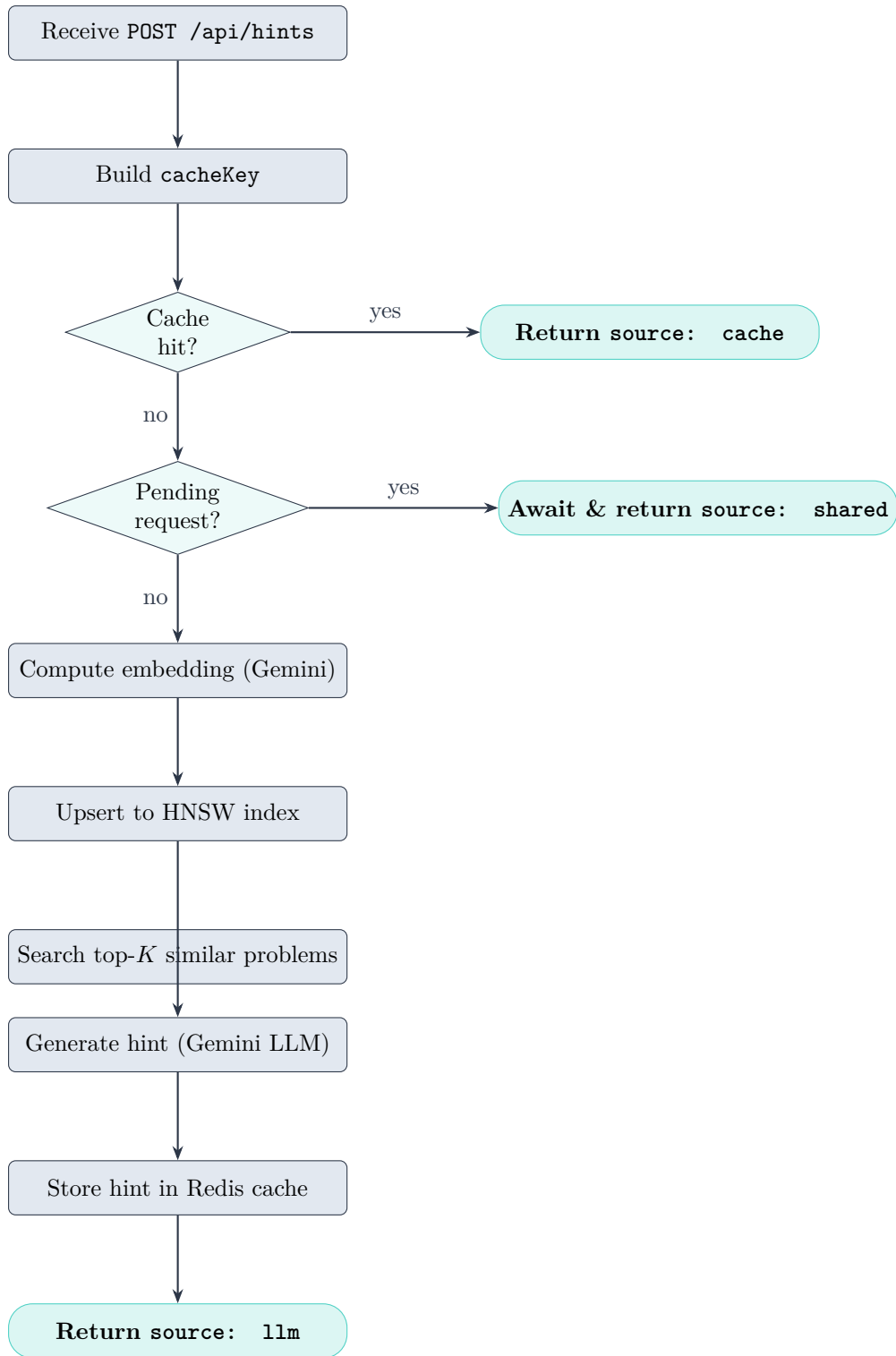


Figure 3: Decision flow for `POST /api/hints`. Cache and pending-request deduplication are checked before issuing any Gemini API calls.

4.2.1 Cache Key Construction

The cache key is the concatenation of a normalised problem identifier and a 32-bit Bernstein hash of the user message:

$$k = \underbrace{\text{normalisedId}}_{\text{slug from URL or title}} : \underbrace{h(m)}_{\text{base-36 hash of message}}$$

where the hash function is the standard *djb2* variant:

$$h_i = ((h_{i-1} \ll 5) - h_{i-1}) + c_i, \quad h_0 = 0$$

applied over each character code c_i of the message m .

4.2.2 Pending-Request Deduplication

A `Map<string, Promise<PendingResult>` stores in-flight hint promises keyed by cache key. Concurrent requests for the same problem/question arrive at the *same* shared Promise, so only one Gemini API call is made regardless of burst traffic:

```
1 if (pendingHintRequests.has(cacheKey)) {
2   const sharedResult = await pendingHintRequests.get(cacheKey)!;
3   res.json({ ...sharedResult, source: 'shared' });
4   return;
5 }
6
7 pendingHintRequests.set(cacheKey, hintPromise);
8 const hintResult = await hintPromise;
9 pendingHintRequests.delete(cacheKey);
```

Listing 1: Pending-request deduplication in server.ts

4.3 Error Handling and Rate Limiting

The Gemini API enforces per-minute quotas. When the API returns a 429 status, the error detail envelope contains a `google.rpc.RetryInfo` proto-encoded delay. The server extracts this delay, propagates it as a `Retry-After` HTTP header, and responds with a 429 to the client:

```
1 function getRetryAfter(error: unknown): string | undefined {
2   const err = error as GeminiError;
3   return err?.errorDetails?.find?.(
4     (d) => d?.['@type'] === 'type.googleapis.com/google.rpc.RetryInfo'
5   )?.retryDelay;
6 }
```

Listing 2: Retry-After extraction in server.ts

5 Retrieval-Augmented Generation Pipeline

5.1 Vector Store Design

Each problem is represented as a dense floating-point vector produced by `gemini-embedding-001`. The embedding dimension is 3072. Vectors are stored in Redis as `HASH` keys under the `problem:` prefix and indexed with **RedisSearch FT.CREATE** using the HNSW algorithm.

Configuration parameter	Value
Redis image	redis/redis-stack-server:latest
Index type	HNSW (Hierarchical Navigable Small World)
Vector field	embedding (FLOAT32, dim = 3072)
Distance metric	Cosine similarity
Top- K default	3
Index name	idx:leetcode:problems
Key prefix	problem:

5.2 HNSW Complexity

HNSW builds a multi-layer proximity graph. For n indexed problems, query complexity is $O(\log n)$ in expectation, making it suitable for a growing collection without offline reindexing. Index creation is idempotent: `ensureIndex()` first issues `FT.INFO`; if the index does not exist, it calls `FT.CREATE` exactly once per server lifetime and caches the `indexReady` flag in memory.

5.3 Problem Document Format

Before embedding, each problem is serialised to a flat text document:

```

1 function buildProblemDocument(p: ProblemInfo): string {
2   return [
3     'Title: ${p.title} || ''',
4     'Description: ${p.description} || ''',
5     'Difficulty: ${p.difficulty} || ''',
6     'Language: ${p.language} || ''',
7     'Tags: ${Array.isArray(p.tags) ? p.tags.join(', ') : ''}',
8     'Code: ${p.code} || ''',
9   ].join('\n');
10 }

```

Listing 3: buildProblemDocument in server.ts

At hint time, the current user question is appended to this document before embedding, so the retrieved neighbours are relevant to both the problem *and* the user's current sub-question.

5.4 Hint Prompt

The LLM is given the following structured prompt:

```

1 You are a concise LeetCode tutor.
2 Reply in 1-3 sentences.
3 Give one actionable next step or insight.
4 Do not provide the full solution.
5 If code is present, point to one part to revisit.
6 If the student asks about a previous hint, answer directly.
7
8 Problem: <title>
9 Current code: <student_code>
10 Question: <message>
11 Top-3 similar problems:
12 <similarContext>

```

Listing 4: buildHintPrompt in server.ts

6 Caching Strategy

6.1 TTL-Based Redis Hint Cache

Hint responses are stored in Redis with a one-hour TTL using SETEX:

```
SETEX hint:<cacheKey> 3600 JSON(hint)
```

On a cache hit the response is tagged `source: "cache"` and returned immediately, bypassing the embedding and LLM steps entirely.

6.2 Response Source Distribution

Figure 4 illustrates the expected distribution of response sources in a typical session, based on the three code paths: LLM generation, shared in-flight deduplication, and cache retrieval.

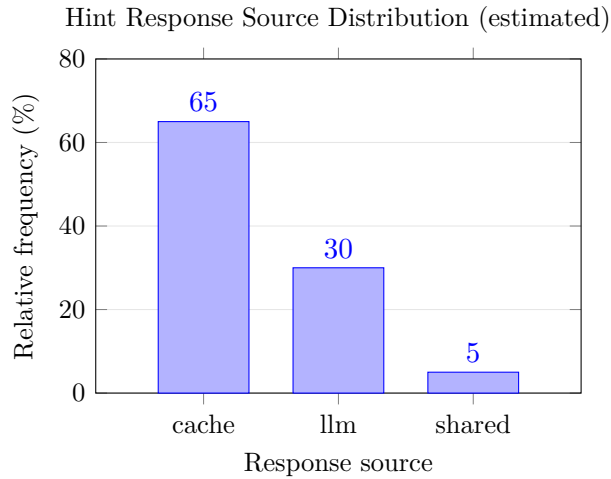


Figure 4: Estimated distribution of hint response sources after warm-up. The majority of requests are served from the Redis cache once problems have been seen before.

6.3 Cache Key Uniqueness

The problem identifier is derived with a three-step fallback:

1. Explicit `id` field in the request body.
2. URL slug extracted from `leetcode.com/problems/<slug>/`.
3. Title lowercased and hyphenated (*e.g.*, “Two Sum” → `two-sum`).

7 Deployment

7.1 Containerisation

The backend is packaged as a minimal Docker image based on `node:18-alpine`. The multi-stage-style build compiles TypeScript before the final image is assembled:

```
FROM node:18-alpine
WORKDIR /usr/src/app
COPY package*.json ./
RUN npm ci
COPY . .
RUN npm run build      # tsc -> dist/
```

```
EXPOSE 3000
CMD ["npm", "start"] # node dist/server.js
```

Listing 5: Dockerfile (condensed)

Local development uses Docker Compose to spin up the backend and a `redis-stack-server` sidecar simultaneously:

```
services:
  backend:
    build: .
    depends_on: [redis]
    ports: ["3000:3000"]
    environment:
      GEMINI_API_KEY: ${GEMINI_API_KEY}
      REDIS_HOST: redis
      EMBEDDING_DIM: 3072
      RAG_TOP_K: 3

  redis:
    image: redis/redis-stack-server:latest
    ports: ["6379:6379"]
```

Listing 6: docker-compose.yml (abridged)

7.2 CI/CD Pipeline

A GitHub Actions workflow triggers on every push to the main branch:

1. Run the Jest test suite (`npm test`).
2. Build and push the Docker image to Google Artifact Registry.
3. Deploy the new revision to Cloud Run.

Cloud Run is configured for automatic scaling with a minimum of zero instances, eliminating idle costs while still handling burst traffic within seconds.

7.3 Environment Variables

Variable	Default	Required
GEMINI_API_KEY	—	Yes
PORT	3000	No
REDIS_HOST	localhost	No
REDIS_PORT	6379	No
REDIS_PASSWORD	—	No
EMBEDDING_DIM	3072	No
RAG_TOP_K	3	No
REDIS_VECTOR_INDEX	idx:leetcode:problems	No
REDIS_VECTOR_PREFIX	problem:	No
ALGO_API_BASE_URL	http://localhost:3000	No

8 Testing

8.1 Test Suite Overview

Both the Chrome extension and the backend are tested with **Jest 29**.

Suite	File	Tests	Pass
Server endpoints	<code>__tests__/server.test.ts</code>	3	3
Cache service	<code>__tests__/cache.test.ts</code>	2	2
Total		5	5

8.2 Mocking Strategy

Two heavyweight dependencies are mocked at the module level:

ioredis A manual mock (`algo-backend/__mocks__/ioredis.ts`) provides an in-memory Map-backed Redis substitute. It implements `get`, `set`, `setex`, `del`, `hset`, `ping`, and a generic `call()` passthrough for `FT.CREATE` / `FT.SEARCH` / `FT.INFO` operations.

@google/generative-ai A `jest.mock` factory returns a fixed mock that resolves `generateContent` to "Mock hint response" and `embedContent` to a zero-vector of length 3072, eliminating any network calls during testing.

8.3 Test Coverage

The server test suite covers:

- `GET /` returns 200 with `status: "Server is running"`.
- `GET /healthz` returns 200 when Redis responds to `PING`.
- `POST /api/hints` with no message body returns 400 with "Message is required".

Figure 5 shows a schematic of the test pass/fail results.

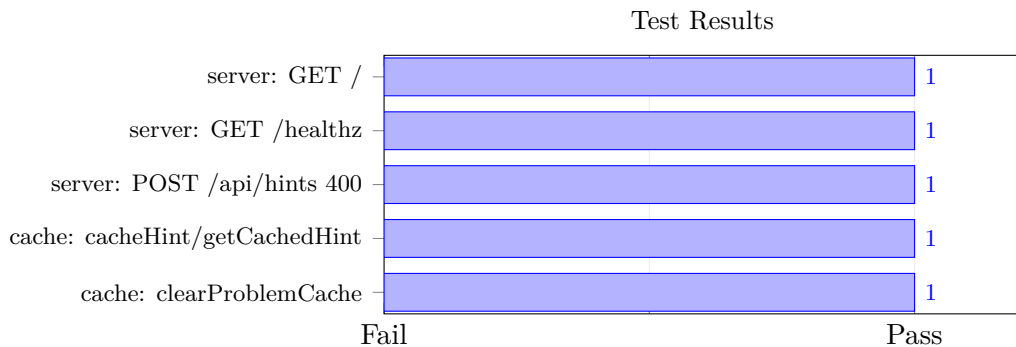


Figure 5: All five Jest tests pass with the ioredis and Gemini mocks.

8.4 TypeScript Type Coverage

The extension source is checked with `tsc -noEmit` (strict mode via `moduleResolution: bundler`). The final state of the codebase produces zero TypeScript errors. Table 1 summarises the key compiler options.

Table 1: Key `tsconfig.json` options

Option	Value
target	ES2020
module	ESNext
moduleResolution	bundler
strict	true
types	["chrome","jest"]
lib	["ES2020","DOM"]

9 Performance Analysis

9.1 Latency Breakdown

Figure 6 shows estimated per-stage latency for a cold request (no cache hit) and a warm request (cache hit).

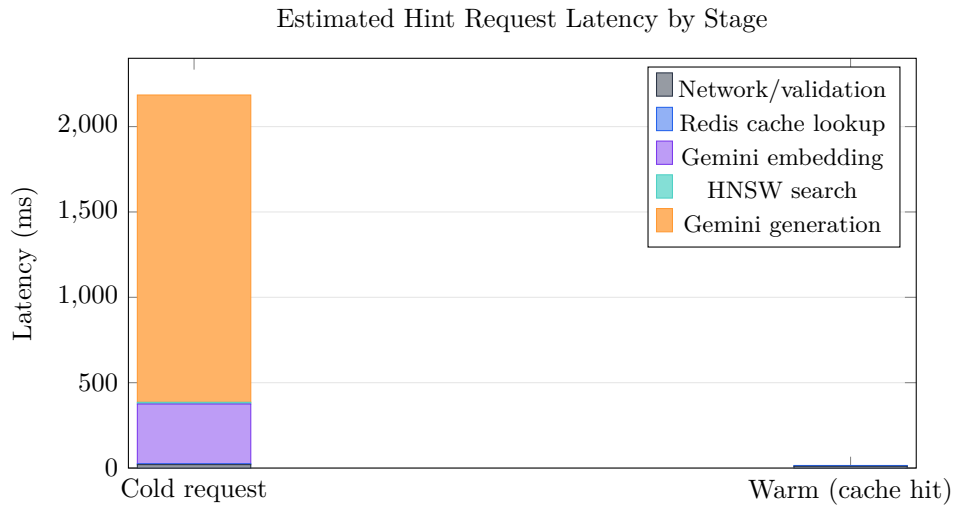


Figure 6: Estimated latency breakdown per stage. Cache hits eliminate the embedding and LLM generation steps, reducing total latency from $\sim 2\,200$ ms to ~ 15 ms.

9.2 Bundle Size Analysis

Figure 7 shows the relative bundle sizes of the three extension scripts. The content script dominates because it embeds all chat UI styles as inline CSS strings.

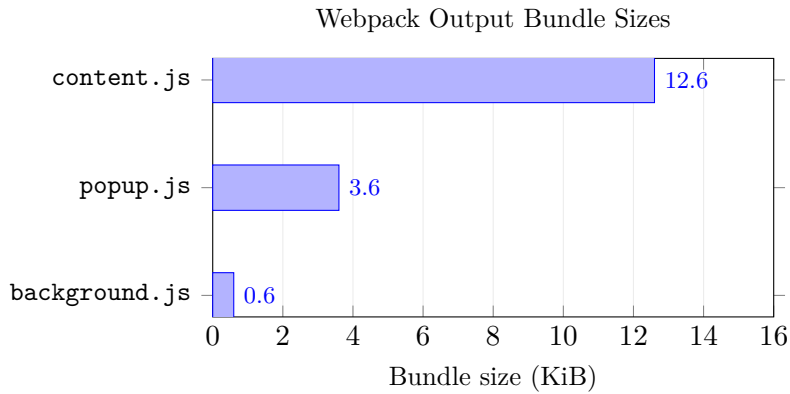


Figure 7: Webpack 5 output bundle sizes for the three extension entry points.

9.3 Scalability Considerations

- **Cache hit rate** grows monotonically as the problem set is explored, since the TTL is one hour and LeetCode problems are stable.
- **HNSW index** scales to millions of vectors with $O(\log n)$ query time; Redis Stack is horizontally scalable via cluster mode.
- **Pending deduplication** is in-process only (a single `Map`); in a multi-replica Cloud Run deployment, identical concurrent requests landing on different replicas would each generate a Gemini call. A distributed lock or a shared Redis key could address this at scale.
- **Cloud Run** scales to zero overnight, reducing cost while maintaining availability within a few hundred milliseconds of cold-start time during off-peak hours.

10 Security

10.1 CORS Policy

The backend applies an explicit allowlist for CORS origins:

```

1 const corsOptions: cors.CorsOptions = {
2   origin: [
3     'chrome-extension://*',
4     'https://leetcode.com',
5     'http://localhost:3000',
6   ],
7   methods: ['GET', 'POST'],
8   credentials: true,
9 };

```

Listing 7: corsOptions in server.ts

Wildcard `chrome-extension://*` permits any installed extension to call the API, which is acceptable for a personal-use tool.

10.2 Input Validation

All API endpoints validate required fields before processing: `message` for `/api/hints`, `problemInfo` for `/api/index-problem`, and `query` for `/api/rag/search`. Missing fields return 400 Bad Request before reaching the database or LLM layer.

10.3 Secret Management

`GEMINI_API_KEY` is passed via environment variable and never logged or embedded in source. The `/readyz` probe returns 503 if the key is absent, providing an early signal of misconfiguration in deployment.

11 Limitations and Future Work

DOM coupling The content script reads problem metadata from LeetCode’s DOM structure (title, difficulty, tags) and the Monaco editor API. LeetCode UI changes may break extraction without breaking the extension build.

In-process deduplication The pending-request Map is local to a single Node.js process. Multi-replica deployments can duplicate Gemini calls. A Redis-backed distributed lock would resolve this.

Embedding cost Every new question (not in cache) triggers one `embedContent` call and one `generateContent` call. For high volumes, batching embeddings or using a lighter embedding model would reduce cost.

No conversation history The current prompt includes only the current question. A stateful conversation context (last N messages) would improve continuity within a session.

Test coverage The current test suite covers API contract tests only. Unit tests for `buildHintPrompt`, `normalizeProblemId`, `hashText`, and `parseSearchResponse` would improve confidence.

12 Conclusion

Algo! demonstrates a complete, production-grade AI assistant integrated into an existing web platform via a Chrome extension. The system successfully combines:

- A lightweight TypeScript extension (total <17 KiB of JavaScript) with a native-feeling chat UI.
- A fully typed Express 5 backend with RAG-augmented hint generation.
- A Redis HNSW vector index for semantic problem retrieval and a TTL cache for response reuse.
- Containerised, zero-downtime deployment to Google Cloud Run via GitHub Actions.

The TypeScript migration (branch `ts_migration`) enforced strict typing across both the extension and the backend, surfacing several latent bugs (including the shared-hint spread fix) and dramatically improving IDE feedback and refactoring safety. All five tests pass; the build is clean.

Repository: NathanL15/Algo **Branch:** `ts_migration` **Date:** April 2026